



Cairo University – Faculty of Engineering

Credit Hours System –Spring 2026

Front-End (Web) – Phase 1 Report

Prepared By:

Name	ID
Ahmad Hisham	1210196
Ahmed Ali	4200368
Mayar Ayman	1210085
Omar Nasser	1210267
Seif Allah Alaa	1200324

1. Task Division & Functional Packages

The Front-End (Web) team is organized into **five functional work packages**, ensuring clear ownership, minimal overlap, and smooth integration with Back-End and Mobile teams.

FE Package	Related Modules	Scope	Assigned Member
FE-1: Authentication & Application Shell	1 + Core Structure	Login, register, account recovery UI; OAuth buttons; global layout (navbar/sidebar); routing and protected routes; role-based UI gating (Artist / Listener)	Seif Allah Alaa
FE-2: Profile & Social Graph	2 + 3	Profile pages (view/edit), avatar upload, social links, privacy settings; followers/following lists; follow/unfollow UI; suggested users; block/unblock UI and blocked list	Ahmad Hisham
FE-3: Tracks, Sets & Playlists + Premium UI	4 + 7 + 12 (UI side)	Playlist/set list & details pages; create/edit/delete playlist/set; add/remove tracks; playlist privacy and share/embed UI; premium feature UI (visuals only)	Omar Nasser
FE-4: Playback & Engagement	5 + 6	Player UI (play/pause/seek/volume); sticky player bar; track details page; likes/reposts UI & counters; comments UI with timestamped input; listening history/recently played UI	Mayar Ayman
FE-5: Discovery, Messaging, Notifications & Admin	8 + 9 + 10 + 11	Home feed, search, trending/charts pages; messaging UI (inbox/thread), track sharing, link previews; notifications list, badge, mark-as-read; admin dashboard UI (reports, hide/remove content, suspend accounts); basic analytics dashboard UI (data from backend)	Ahmed Ali

2. Front-End Code Style (React + TypeScript)

Language & Tooling:

- React + TypeScript for Web Front-End.
- Prettier as the single source of formatting truth; ESLint integrated.
- Pre-commit hooks (Husky + lint-staged) enforce style before push.

Formatting Rules:

- All code must be formatted using Prettier.
- Indentation, line wrapping, and spacing enforced consistently.

Naming Conventions:

- React components: PascalCase (e.g., UserProfile.tsx)

- Custom hooks: useXxx (e.g., useAuth())
- Functions/variables: camelCase (e.g., fetchPlaylists)
- Types/interfaces: CamelCase (e.g., User, Playlist)
- Constants: CamelCase (e.g., maxSize)

File & Folder Conventions:

- Folders: kebab-case (components/common/)
- Component files: PascalCase.tsx (TrackCard.tsx)
- Helpers: camelCase.ts (formatTime.ts)

Project Structure:

```
src/
components/  # reusable UI components
  common/    # shared components (Button, Modal, LoadingState)
  <feature>/ # feature-specific UI (profile, playlists)
pages/       # route-level screens
services/    # API clients (HTTP calls)
mocks/       # mock services for backend simulation
hooks/       # reusable hooks
types/       # shared TypeScript types/interfaces
utils/       # stateless helpers/constants
routes/      # route definitions
```

TypeScript Rules:

- Avoid any; if used, justify with a code comment.
- Prefer explicit types at module boundaries (API requests, component props, global state).
- Consistently use interface or type per team standard.

Component Guidelines:

- Small, focused, presentational components.
- Data fetching / side effects in services/ + hooks/.
- Pages orchestrate fetching and pass data down as props.

API Calling Convention:

- No HTTP calls in UI components.
- All requests go through services/ (e.g., authService, playlistService).
- Standardized error handling: user-friendly messages in UI, technical logs in development only.

Git & Review:

- Feature branches only (no direct commits to main).
- Branch naming: fe/<feature>/<short-desc>.
- PR must include description and screenshots (if applicable).

3. System Architecture (Frontend Context)

Main Architectural Choices:

- Component-Based Architecture with separation of concerns.
- React Router for routing.
- Service layer for API calls.
- Mock services for backend simulation (controlled via environment variable).
- Reusable UI components and responsive design.
- Protected routes for role-based access.
- Global state for authentication, notifications, and player state (Redux Toolkit or React Context).
- Local UI state managed via useState / useReducer.

Design Patterns Utilized:

1. **Observer Pattern:** React state updates, Redux for global audio player state.
2. **Dependency Injection:** Switch easily between real API and mock services.
3. **Container/Presentational Pattern:** Separating data fetching from UI rendering for testability and reuse.

4. Pages

Page	URL	Notes
Login page	/login	Auth Page
Register page	/register	Auth Page
Home/Feed page	/	Discovery & Stream Feed
Profile page	/profile/:id	Profile & Social Graph
Playlist page	/playlists/:id	Tracks/Sets & Playlists

Track Details page	/tracks/:id	Playback & Engagement
Notifications page	/notifications	Notifications UI
Messages page	/messages	Messaging UI
Upload page	/upload	Upload UI
Subscription/Pricing page	/premium	Premium Features
Admin Dashboard	/admin	Admin & Analytics